॥वसुधैव कुटुम्बकम्॥

SYMBIOSIS INSTITUTE OF TECHNOLOGY, PUNE

Symbiosis International (Deemed University)

(Established under section 3 of the UGC Act, 1956)

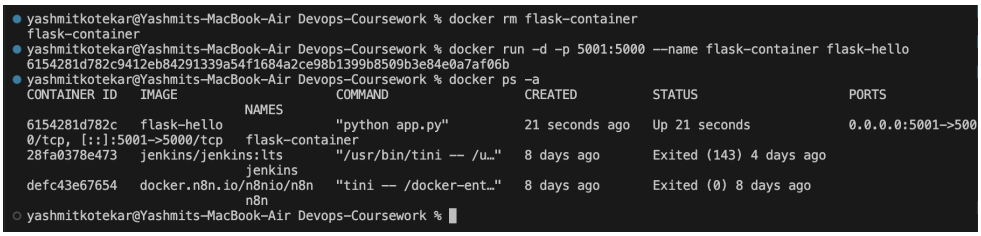
Re-accredited by NAAC with 'A' grade (3.58/4) | Awarded Category – I by UGC

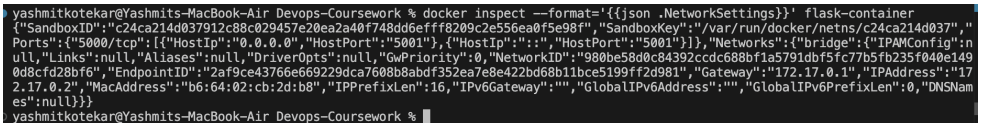
Founder: Prof. Dr. S. B. Mujumdar, M. Sc., Ph. D. (Awarded Padma Bhushan and Padma Shri by President of India)

Assignment No. 09

Subject:	DevOps Lab
TCODE:	TE7950
Name of Student	**Yashmit Kotekar**
PRN No.	**23070122236**
Branch	CSE, Batch (2023-27), L4
Academic Year & Semester	25 – 26, Sem VII
Date of Performance	23rd July, 2026
Title of Assignment:	Container Management & Networking
Theory:	Docker container management is the process of creating, running, monitoring, stopping, inspecting, and removing containers. Port mapping allows applications running inside a container to be accessed from the host machine by mapping a host port to a container port. Docker provides commands such as docker ps to view running containers, docker inspect to examine detailed container configuration and network settings, docker stop to gracefully stop a container, and docker rm to remove unused containers. These commands help efficiently manage the lifecycle of Docker containers and verify their network configuration and status. Commands Used • Created and started the Docker container in detached mode with port mapping: docker run -d -p 8081:5000 --name flask-network-demo my-flask-app • Listed all running Docker containers: docker ps • Stopped the running Docker container: docker stop flask-network-demo • Listed all Docker containers, including stopped containers: docker ps -a • Inspected the container's network configuration and settings: docker inspect flask-network-demo • Removed the stopped Docker container: docker rm flask-network-demo

	• Verified that the container had been removed: docker ps -a
Implementation:	**Step 1: Run a Container with Port Mapping** Started Docker Container — Started the Docker container named flask-network-demo in detached mode and mapped host port 8081 to container port 5000, allowing the application to run in the background. **Step 2: Access the Application via Your Browser** Verified Application Access — Accessed the Dockerized Flask application through http://localhost:8081 and verified that the application was successfully running inside the Docker container. **Step 3: List Running Containers** Listed Running Containers — Displayed all currently running Docker containers using the docker ps command and verified that the flask-network-demo container was running with port mapping from 8081 on the host to 5000 inside the container. **Step 4: Stop the Running Container** Stopped Docker Container — Stopped the running flask-network-demo container using the docker stop command, terminating the application gracefully. **Step 5: List Running and All Containers** Verified Container Status — Used the docker ps -a command to display all Docker containers and verified that the flask-network-demo container had stopped successfully and was in the Exited state. **Step 6: Inspect the Container's Network Settings** Inspected Container Network Settings — Used the docker inspect command to view detailed information about the flask-network-demo container, including its NetworkSettings and network configuration. **Step 7: Remove the Stopped Container** Removed Docker Container — Removed the flask-network-demo container using the docker rm command and verified its successful removal by confirming that it no longer appeared in the list of Docker containers.





Subject Faculty Name: Prof. Arijit Dutta